

Unit 2 Case Studies

Hypervisor Case Studies

Case Study: Microsoft – WSL 2 with Hyper-V (Type 1: Bare-Metal)

Context: Microsoft Windows operating system for developer workstations across enterprises and independent developers. WSL 2 was released with Windows 10 version 2004 (released May 2020) and is widely deployed globally for cross-platform development.

Problem: Developers working on Windows needed a seamless way to run Linux-based tools, containers, and development environments without sacrificing performance or maintaining virtual machines manually. Previous solutions (WSL 1) had compatibility and performance limitations, forcing developers to either dual-boot systems, use separate VM tools, or maintain complex setups.

Solution: Microsoft implemented WSL 2 using Hyper-V as a lightweight, bare-metal hypervisor running a specialized Linux kernel distribution. WSL 2 creates a utility VM that:

- Manages the Linux kernel directly at the hypervisor level
- Provides native system call compatibility for full Linux applications
- Allows automatic kernel servicing through Windows Update (like hardware drivers)
- Integrates seamlessly with Windows file systems and applications
- Uses hardware virtualization features for minimal overhead

The architecture enables developers to invoke Windows applications from Linux terminals and vice versa, supporting containerized workflows and DevOps tooling natively.

Results/Lessons: WSL 2 achieved rapid adoption, becoming the standard development environment for thousands of enterprises migrating to cloud-native development practices. Performance benchmarks showed near-native Linux execution speeds compared to WSL 1. Developer feedback highlighted significant productivity gains from eliminating context-switching between dual-boot systems or separate VMs. Microsoft successfully unified the developer experience across Windows and Linux ecosystems, establishing WSL as a critical tool in the broader Windows ecosystem modernization strategy.

Reference: <https://devblogs.microsoft.com/commandline/wsl2-will-be-generally-available-in-windows-10-version-2004/>

Case Study: Oracle – VirtualBox Desktop Virtualization (Type 2: Hosted)

Context: Oracle VirtualBox is a cross-platform, open-source Type 2 hypervisor running on Windows, macOS, Linux, and Solaris host systems. It serves millions of individual

developers, enterprises, educational institutions, and research organizations globally for desktop and server virtualization since its acquisition by Oracle.

Problem: Organizations needed a cost-effective, vendor-neutral virtualization solution for desktop and laboratory environments that did not require expensive licensing or proprietary lock-in. Researchers, students, and development teams required the ability to run multiple operating systems on single workstations for testing, sandboxing, and cross-platform development without dedicated server infrastructure.

Solution: VirtualBox implements Type 2 hypervisor architecture where:

- The hypervisor runs as a hosted application on top of the Windows, Linux, or macOS host OS
- Guest VMs share the host OS kernel infrastructure and resource management
- Virtual machines are managed through a user-friendly graphical interface
- Snapshots and cloning enable rapid environment provisioning
- VirtualBox remains completely free and open-source, eliminating licensing costs
- Cross-platform compatibility allows consistent workflows across different host operating systems

The hosted approach prioritized ease of setup and accessibility over performance, making it suitable for development, testing, and educational environments where maximum performance isn't critical.

Results/Lessons: VirtualBox achieved widespread adoption in education (universities worldwide use it in computer science programs), open-source development communities, and SMB enterprises. Its combination of free licensing, cross-platform support, and ease of use established it as the de-facto standard for affordable desktop virtualization. Organizations reported significant cost savings compared to proprietary solutions, while maintaining flexibility for rapid environment changes and disaster recovery through snapshot technology. The open-source model enabled community contributions that continuously improved compatibility and features.

Reference: <https://www.virtualbox.org/>

Case Study: AWS – Nitro Hypervisor for EC2 (Type 1: Bare-Metal)

Context: Amazon Web Services (AWS) operates the largest public cloud infrastructure globally, serving millions of customers across enterprises, startups, and government organizations. AWS developed the Nitro System—a custom bare-metal hypervisor—to power Amazon EC2 instances, the foundational compute service of AWS with over 1000 instance types.

Problem: As AWS scaled from handling thousands to millions of EC2 instances, traditional hypervisors (Xen) created performance overhead and resource inefficiencies. Each hypervisor instance consumed CPU cycles, memory, and network bandwidth that couldn't be

used by customer workloads. AWS needed a way to maximize resource utilization, reduce virtualization overhead, and support increasingly demanding workloads (AI/ML training, high-performance computing, real-time analytics) while maintaining security and multi-tenancy guarantees.

Solution: AWS designed and implemented Nitro, a custom bare-metal hypervisor that:

- Offloads virtualization functions to specialized hardware (Nitro cards) rather than consuming host CPU
- Reduces hypervisor overhead from ~20% of CPU resources to <5%, freeing resources for customer workloads
- Provides hardware-enforced isolation between customer instances and management systems
- Supports consistent, high-speed networking (400 Gbps Ethernet on latest generations)
- Enables tight integration with AWS services (EBS, VPC, CloudWatch) through dedicated hardware paths
- Maintains security and compliance certifications (FedRAMP, HIPAA, PCI-DSS)

Nitro's bare-metal design pushed virtualization logic to dedicated hardware controllers, allowing the host CPU to run customer code exclusively.

Results/Lessons: Nitro hypervisor enabled AWS to maintain competitive pricing while offering superior performance and enabling new instance types for specialized workloads (bare-metal instances, GPU acceleration, HPC-optimized instances). The architecture scaled to support AWS's exponential growth—by 2024, Nitro powers millions of concurrent instances across global regions. Performance improvements from Nitro directly translated to lower customer TCO and enabled latency-sensitive applications (real-time bidding, high-frequency trading, multiplayer gaming) to run efficiently on AWS. AWS's investment in custom hypervisor technology became a competitive differentiator, as documented in customer case studies across financial services, media & entertainment, and scientific research sectors.

Reference: <https://aws.amazon.com/ec2/>

Case Study: Microsoft – Hyper-V Enterprise Virtualization (Type 3: Hybrid)

Context: Microsoft Hyper-V is deployed across millions of Windows Server instances in enterprise data centers worldwide. Unlike WSL 2 which focuses on developer workstations, Hyper-V powers production infrastructure for financial institutions, healthcare organizations, e-commerce platforms, and government agencies. It can operate as both a bare-metal hypervisor (Hyper-V Server standalone) and hosted hypervisor (running as a role within Windows Server operating system).

Problem: Large enterprises required a flexible virtualization platform that could adapt to diverse infrastructure requirements. Some organizations needed dedicated bare-metal hypervisors for maximum performance in critical production environments, while others preferred hosted deployments to leverage existing Windows Server management tooling and Active Directory integration. Manual context-switching between different hypervisor technologies increased operational complexity and training costs. Organizations also needed seamless migration paths between on-premises infrastructure and Azure cloud services.

Solution: Microsoft designed Hyper-V as a hybrid hypervisor architecture supporting both deployment modes:

Bare-Metal Mode (Hyper-V Server):

- Standalone hypervisor with minimal Windows Server footprint
- Direct hardware access for optimal performance
- Manages VMs through Remote Server Administration Tools (RSAT)

Hosted Mode (Windows Server Hyper-V Role):

- Hypervisor integrated as a Windows Server role
- Host OS (management partition) runs as first-class guest alongside workload VMs
- Unified management through Server Manager and System Center
- Integration with Active Directory, Group Policy, and Windows authentication
- Shared infrastructure for both host management and VM hosting

The hybrid approach allowed organizations to choose deployment based on workload requirements while maintaining unified management tools and operational procedures across both modes. Hyper-V maintains consistency in VM formats, networking configuration, and management interfaces whether deployed bare-metal or hosted.

Results/Lessons: Hyper-V captured significant market share in Windows Server environments (Gartner estimates ~60-70% of on-premises virtualization deployments in Windows Server shops). Enterprises valued the operational flexibility and reduced training overhead from unified tooling. Organizations reported smooth migration paths from hosted to bare-metal deployments as performance requirements increased, without VM format conversion. The hybrid architecture enabled Microsoft to capture organizations at different maturity stages—smaller deployments used hosted mode for cost efficiency, while high-performance environments transitioned to bare-metal mode without vendor lock-in concerns. Hyper-V became a strategic advantage for Azure adoption, as organizations extended their on-premises Hyper-V infrastructure to Azure Stack and Azure VMware Solution, creating a consistent hybrid cloud model.

Reference: <https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/hyper-v-on-windows-server>

Popek-Goldberg Principles and x86 Architecture Implementation

Case Study: Intel – VT-x Hardware Virtualization Extensions (2005)

Context: The x86/IA-32 architecture lacked native support for efficient virtualization throughout the 1990s and early 2000s. Intel recognized the growing demand for virtualization in enterprise data centers and academic research, particularly as the open-source movement expanded. The company committed to adding hardware-level virtualization support to future processor generations, resulting in the development of VT-x (Vanderpool).

Problem: When evaluated against the Popek-Goldberg virtualization requirements (formulated in 1974), the x86 architecture failed Theorem 1, which states that all sensitive instructions must be privileged. The IA-32 instruction set contained 18 sensitive, unprivileged instructions (such as POPF, PUSHF, SGDT, SIDT, SLDT, SMSW, LAR, LSL, VERR, VERW, and others related to segment registers and protection system access). This design flaw forced virtualization software to use complex workarounds like binary translation (code rewriting at runtime) and shadow page tables, which introduced significant performance overhead. Organizations running virtualized workloads on x86 systems experienced 15-30% performance degradation compared to native execution.

Solution: Intel designed VT-x (Virtual Machine Extensions) as a hardware-assisted virtualization solution that directly addressed Popek-Goldberg's requirements. Released in November 2005 on Pentium 4 processors (Models 662 and 672), VT-x introduced:

- 13 new machine instructions (VMPTRLD, VMPTRST, VMCLEAR, VMREAD, VMWRITE, VMCALL, VMLAUNCH, VMRESUME, VMXOFF, VMXON, INVEPT, INVVPID, and VMFUNC) to manage virtual machine execution
- A new execution mode where guest operating systems perceive themselves as running at full privilege (ring 0) while the host OS remains protected, satisfying Popek-Goldberg's equivalence and resource control properties
- VMCS (Virtual Machine Control Structure) mechanism to store guest CPU state and control parameters
- Extended Page Tables (EPT) support in later generations (Nehalem, 2008) to enable efficient memory virtualization by allowing hardware to directly translate guest-physical to host-physical addresses, eliminating shadow page table overhead

The solution transformed unprivileged sensitive instructions into operations that trap to the hypervisor naturally, converting the x86 architecture into a classically virtualizable system as defined by Popek-Goldberg Theorem 1.

Results/Lessons: VT-x enabled the x86 architecture to meet formal virtualization requirements decades after its initial design, fundamentally transforming the virtualization landscape. By 2015, nearly all new Intel server, desktop, and mobile processors included

VT-x support (with Intel Atom as the primary exception). Performance overhead for VT-x equipped systems dropped from 20-30% to approximately 2-5% for most workloads—approaching the theoretical limits described in Popek-Goldberg analysis. This enabled x86 to become the dominant platform for enterprise virtualization, cloud infrastructure, and data center consolidation. VMware, Red Hat, and other hypervisor vendors rapidly adopted VT-x, moving away from pure software-based binary translation toward hybrid approaches leveraging hardware support. Intel's success motivated AMD to develop competing technology (AMD-V), establishing an industry standard that drove virtualization adoption across the entire x86 ecosystem.

Reference: [https://en.wikipedia.org/wiki/X86_virtualization#Intel_virtualization_\(VT-x\)](https://en.wikipedia.org/wiki/X86_virtualization#Intel_virtualization_(VT-x))

Case Study: VMware – Software-Based Binary Translation for x86 Virtualization (1990s-2000s)

Context: VMware, founded in 1998, pioneered commercial x86 virtualization when the architecture was fundamentally non-virtualizable according to Popek-Goldberg standards. The company developed innovative software techniques to work around x86's architectural limitations, enabling full virtualization on commodity hardware when hardware support did not exist. Between 1999-2005, VMware's products (ESX Server and Workstation) became the industry standard for x86 virtualization, capturing significant market share before Intel and AMD added hardware support.

Problem: The x86 architecture violated Popek-Goldberg Theorem 1 because critical instructions that could compromise the hypervisor remained unprivileged. Classic trap-and-emulate virtualization (relied upon by mainframe systems like IBM System/360) was impossible because problematic instructions executed silently without generating traps. For example, POPF (Pop Flags) could modify the interrupt flag without triggering a trap if executed in user ring, allowing a guest OS to inadvertently disable interrupts and destabilize the entire system. Similarly, segment-related instructions (SGDT, SIDT, LAR, LSL, etc.) allowed guests to access or modify segment descriptors, potentially breaking isolation boundaries. VMware needed to virtualize x86 protected mode efficiently despite these architectural deficiencies, requiring research and innovation to meet Popek-Goldberg's three criteria (equivalence, resource control, and efficiency) through software-only techniques.

Solution: VMware developed three complementary software virtualization techniques that collectively satisfied Popek-Goldberg requirements without hardware support:

1. **Binary Translation:** At runtime, VMware's hypervisor identified and rewrote problematic (critical) instructions before guest code execution. For instance, POPF instructions were translated into equivalent sequences of ring 3 (user-mode) instructions that could safely modify flags. This approach required building a coherent translation cache that detected code patching, self-modifying code, and page reuse by the guest OS to avoid stale translations.

2. **Shadow Page Tables:** Because x86 grants guest operating systems direct access to memory management hardware (the MMU), VMware maintained parallel "shadow" page tables that intercepted and emulated all memory mapping changes. When a guest OS modified its page tables, traps were triggered, allowing VMware to synchronize the shadow tables and maintain isolation. This technique prevented guests from accessing other VMs' memory while preserving the illusion that each guest controlled the full memory hierarchy.
3. **Descriptor Table Shadowing:** The x86 architecture uses hidden processor state to cache segment descriptors. Once loaded, descriptor contents cannot be read back from the processor. VMware maintained shadow descriptor tables to track segment descriptor modifications by guest operating systems, ensuring that guest descriptor manipulations didn't compromise the hypervisor's isolation.

Despite software-only virtualization, VMware achieved near-native performance through aggressive caching of translated code blocks and strategic optimization of the most frequently executed instruction sequences. Their research (published by Keith Adams and Ole Agesen at ASPLOS 2006) demonstrated that well-tuned binary translation could achieve comparable performance to first-generation hardware virtualization, showing that software flexibility sometimes offset hardware rigidity.

Results/Lessons: VMware's software-based virtualization proved that an architecture failing Popek-Goldberg's formal requirements could still be virtualized efficiently if developers employed creative software techniques. This established VMware as the market leader in x86 virtualization throughout the 1990s and 2000s, capturing enterprises that needed reliable multi-tenant virtualization. The company's success demonstrated massive commercial demand for x86 virtualization, motivating Intel and AMD to invest in hardware extensions. Even after Intel VT-x and AMD-V became available (2005-2006), VMware continued using binary translation in parallel with hardware virtualization, leveraging its software flexibility to support older processors and optimize specific workloads. VMware's technical achievements proved Popek-Goldberg's requirements were sufficient but not necessary for practical virtualization, establishing a precedent that influenced hypervisor design across the industry. The research fundamentally changed how the computing community viewed virtualization, demonstrating that cleverly designed software could overcome architectural limitations.

Reference: https://en.wikipedia.org/wiki/X86_virtualization#Software-based_virtualization

Case Study: Linux Community – KVM and the Adoption of Hardware-Assisted Virtualization (2006-2007)

Context: The Linux kernel community pursued open-source virtualization by leveraging the newly available Intel VT-x and AMD-V hardware extensions that finally brought x86 architecture into compliance with Popek-Goldberg virtualization requirements. Avi Kivity began developing KVM (Kernel-based Virtual Machine) at Qumranet in mid-2006, releasing it in October 2006, and successfully merging it into the mainline Linux kernel version 2.6.20

on February 5, 2007. This represented a watershed moment where Popek-Goldberg's formal requirements directly enabled practical, efficient virtualization without the complexity of binary translation.

Problem: Despite VMware's impressive software-based virtualization, the Linux community recognized that supporting hardware virtualization extensions directly in the kernel would be more efficient and maintainable than implementing complex binary translation and shadow page table machinery. The prior era of software-only virtualization required constant optimization and careful handling of edge cases. The availability of Intel VT-x (November 2005) and AMD-V (May 2006) provided hardware-level support for the Popek-Goldberg requirements, making trap-and-emulate virtualization finally possible on x86. However, this required kernel integration to expose virtualization capabilities to user-space VMMs. The Linux community needed a lightweight, efficient kernel module that could leverage hardware virtualization for standard Linux deployments running on commodity servers and developer workstations.

Solution: KVM was architected as a lightweight kernel module that:

- Exposed the `/dev/kvm` interface, allowing user-space applications (primarily QEMU) to directly leverage Intel VT-x and AMD-V hardware extensions
- Delegated CPU and memory virtualization to hardware (satisfying Popek-Goldberg's efficiency requirement through direct hardware execution of guest instructions)
- Managed guest VM address spaces and state transitions using VT-x VMCS (Virtual Machine Control Structure) or AMD-V equivalents
- Delegated device emulation and I/O handling to user-space (QEMU), maintaining a clear separation between kernel virtualization and user-space peripheral simulation
- Supported paravirtualization through VirtIO (Virtual I/O), allowing guests to achieve near-native I/O performance by collaborating with the host through optimized I/O interfaces

By directly leveraging hardware extensions that satisfied Popek-Goldberg's theoretical requirements, KVM avoided the complexity of binary translation and shadow tables while achieving excellent performance. The design philosophy recognized that hardware virtualization extensions, once available, eliminated the need for complex software workarounds, simplifying the hypervisor and making it more maintainable.

Results/Lessons: KVM's integration into the mainline Linux kernel democratized virtualization by making it freely available to all Linux users without requiring proprietary hypervisors like VMware ESX. By 2010, KVM had become the standard hypervisor for Linux deployments, eventually displacing Xen (another open-source hypervisor) as the preferred solution. Red Hat's 2008 acquisition of Qumranet (KVM's originating company) signaled the enterprise viability of hardware-assisted virtualization, accelerating adoption across the industry. KVM demonstrated that Popek-Goldberg's formal requirements, once satisfied by hardware, enabled simpler, more efficient hypervisor implementations than software-only approaches. The success of KVM validated Intel and AMD's investment in hardware virtualization extensions as the right architectural direction. Today, KVM serves as the

foundation for cloud platforms including OpenStack, OpenNebula, and numerous managed hosting providers, proving that Popek-Goldberg compliance through hardware support scales from single developer machines to global cloud infrastructure. The case demonstrates how formal theoretical frameworks (Popek-Goldberg principles) guide industrial technology decisions, and how hardware vendors' willingness to address these theoretical requirements enables entire ecosystems of virtualization technologies to flourish.

Reference: https://en.wikipedia.org/wiki/Kernel-based_Virtual_Machine

VM Migration - Live (Pre-copy and Post-copy)

Case Study: XenServer – Pre-Copy Live Migration Architecture

Context: As virtualization adoption grew, organizations using the Xen hypervisor needed a method to perform hardware maintenance, cluster load balancing, and proactive disaster response without causing downtime for critical applications running on VMs.

Problem: Traditional "cold" migration required the VM to be completely suspended, its state transferred to the target host, and then restarted. This resulted in significant service blackout (often several seconds or minutes depending on memory size) that was unacceptable for active databases, network servers, and real-time systems. Administrators needed a way to move a running VM across physical hosts seamlessly.

Solution: The Xen research team (and later Citrix XenServer) pioneered early implementations of "Live Migration" leveraging an iterative pre-copy approach. Their strategy addressed the challenge through specific phases:

1. **Iterative Pre-copy phase:** The Xen hypervisor copies pages of active memory from the source host to the destination host while the VM continues executing.
2. **Dirty Page Tracking:** Xen employs shadow page tables to intercept and track subsequent writes (creating "dirty pages") to memory that has already been copied to the destination.
3. **Successive Rounds:** In each subsequent round, the hypervisor sends only the pages dirtied in the previous round. As the active working set of memory is generally smaller than total memory, the amount of data transferred typically shrinks each round.
4. **Stop-and-copy (Blackout):** When the dirty page generation rate matches the network transfer rate (or a fixed iteration limit is reached), Xen briefly pauses the VM, transfers the final CPU registers and remaining dirty pages, and resumes the VM on the target host.

Results/Lessons: The Xen pre-copy migration successfully reduced service downtime to mere milliseconds—short enough to be undetectable to TCP connections, preventing application timeouts. However, the system revealed that highly write-intensive workloads could cause the pre-copy phase to fail to converge, leaving a massive final payload for the blackout phase. This demonstrated that while pre-copy is highly effective for read-mostly

memory workloads, alternative or hybrid strategies (like post-copy) are occasionally needed to bound the maximum blackout time for exceptional workloads.

Reference: <https://dl.acm.org/doi/10.5555/1251203.1251223> (Clark, Christopher; et al. "Live migration of virtual machines")

Case Study: Google Cloud Platform (Compute Engine) – Transparent Maintenance via Live Migration

Context: Google Cloud operates millions of VMs for enterprise customers worldwide. To keep the underlying infrastructure secure and updated (e.g., BIOS updates, patching vulnerabilities like Meltdown/Spectre, or hardware replacement), Google needs to constantly perform host maintenance.

Problem: Cloud customers expect continuous availability (often 99.99% uptime). Forcing customers to handle VM reboots for provider-side hardware maintenance damages reliability and the developer experience. Traditional maintenance blocks often require coordinating downtime across global teams.

Solution: Google Cloud implemented an automated, planetary-scale Live Migration system across Compute Engine. It heavily leverages pre-copy methods coordinated by their cluster management:

1. **Source Brownout:** The VM continues to run on the source host while memory pages are continuously copied to a target host. Google's algorithm tracks changed pages and resends them, explicitly balancing the transfer of memory bytes against the rate of guest modifications.
2. **Blackout:** Once the remaining state reaches diminishing returns in the pre-copy phase, the VM is briefly paused. The remaining state, CPU registers, and device states are transferred.
3. **Target Brownout:** The VM resumes on the target host. To ensure seamless network connectivity during the migration, the source VM's host provides temporary packet forwarding until the network fabric completely updates its routes to the new host.
4. **Time Management:** If the blackout event stretches, the system daemon pauses and synchronizes the system clock to hide the pause from guest applications.

Results/Lessons: Google Compute Engine routinely live-migrates VMs without customer intervention. During major security events, Google has patched its entire global fleet without a single customer reboot. The overall blackout time remains typically under a single second. This case study proves that robust pre-copy live migration, combined with advanced network forwarding, is an absolute prerequisite for hyperscale cloud platforms, shifting the operational burden of physical maintenance entirely from the user to the provider.

Reference: <https://cloud.google.com/compute/docs/instances/live-migration-process>

Case Study: Red Hat / QEMU-KVM – Post-Copy Live Migration for Write-Intensive Workloads

Context: Large enterprise data centers heavily use KVM/QEMU to host massive, critical databases and transactional workloads. Open-source maintainers, including engineers at Red Hat and academic researchers, needed to continuously improve KVM's live migration capabilities.

Problem: The standard iterative pre-copy migration approach completely fails for VMs running extremely write-intensive workloads (like in-memory databases or heavily transacting SQL servers). If the guest operating system dirties memory pages faster than the hypervisor can transmit them over the network, the pre-copy phase never converges to a small enough set of dirty pages. This leads to infinite migration loops, prolonged network utilization, or forced (and unacceptably long) VM blackout times when the system finally gives up and pauses the guest to transfer massive amounts of data.

Solution: The QEMU/KVM community integrated "Post-Copy" live migration (originally researched by Michael Hines et al. and later implemented into QEMU 2.5 in 2015). This alternative strategy shifts the order of operations:

1. **Immediate Blackout & State Transfer:** The hypervisor briefly pauses the VM on the source node and immediately transfers the minimal, essential non-memory state (CPU registers, device state) to the destination.
2. **Resume on Target:** The VM is resumed on the target physical host *before* its memory footprint is transferred.
3. **Demand Paging (Userfaultfd):** As the guest OS resumes execution, it frequently attempts to access memory pages that have not yet been copied over. QEMU uses a Linux kernel feature called `userfaultfd` to intercept these memory misses. The vCPU thread halts, and QEMU demands the specific page over the network from the source host.
4. **Background Transfer:** Concurrently to the on-demand page fetching, the source host continuously pushes all remaining memory pages to the target in the background to ensure completion.

Results/Lessons: The post-copy migration successfully solves the "non-convergence" problem, guaranteeing a very short, deterministic blackout time regardless of how fast the VM is modifying memory. However, the system highlighted a critical reliability trade-off over pre-copy: if a network failure occurs during post-copy, the VM irrecoverably crashes because its state is fundamentally split across two separate physical machines. Additionally, guest performance is significantly degraded during the migration window due to network-induced page faults. Thus, post-copy is primarily deployed as a fallback mechanism when pre-copy iterations declare non-convergence.

Reference: <https://dl.acm.org/doi/10.5555/1618525.1618528> (Hines, M. et al., "Post-copy based live virtual machine migration using adaptive pre-paging and dynamic self-ballooning")